# LELEC2531 – FIRST PROJECT REPORT

**Submission deadline: October 25, 2026 at 23:59.**

Project topic: AI Multiply–Accumulate (MAC) Accelerator – Phase 1 (Weeks 3–6)

| Group no. | Name 1 | Name 2 | Name 3 |
|---|---|---|---|
| *[Group]* | *[Name]* | *[Name]* | *[Name]* |
| **NOMAs:** | *[NOMA 1]* | *[NOMA 2]* | *[NOMA 3]* |

**Q1 – Context (1 pt)**

Multiply–accumulate (MAC) operations, $S_{n+1} = S_n + A_n B_n$, are repeated in neural-network matrix multiplications, convolution layers and dot products. A dedicated MAC accelerator performs the multiply-and-add operation directly in hardware, potentially reducing the processor instruction count and allowing the processor to perform other tasks. This is useful for embedded/edge AI, where throughput and energy efficiency matter. **These advantages are motivations, not yet measured results of our implementation.**

**Q2 – Chosen Topology (1 pt)**

We propose a **signed 16-bit × signed 16-bit MAC** with a **32-bit result**. The 32-bit input word packs $A =$ `input[31:16]` and $B =$ `input[15:0]` (two's complement). The product is optionally arithmetically right-shifted by $s \in [0, 31]$ before being accumulated. The datapath implements

$$S_{n+1} = \begin{cases} S_n + (\mathrm{ASR}_s(A_n B_n)), & \text{accumulate mode,} \\ \mathrm{ASR}_s(A_n B_n), & \text{replace mode.} \end{cases}$$

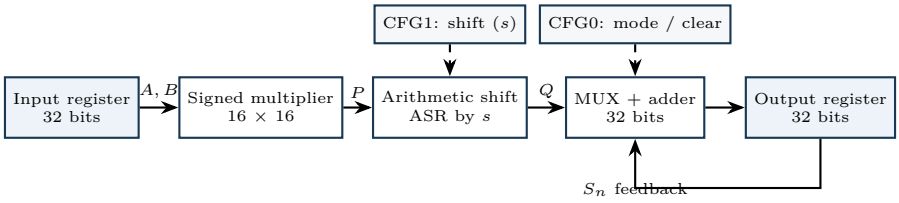



Figure 1. First-version MAC block diagram (proposed implementation).

| | |
|---|---|
| **Input** | 32 bits; writing `wr_input` captures two operands and starts one operation. |
| **CFG0 / CFG1** | Two 8-bit configuration registers: CFG0 bit 0 selects accumulate/replace; writing bit 1 of CFG0 requests a clear pulse (not stored). CFG1 bits [4:0] select shift. |
| **Output** | 32 bits; stores the accumulated result, with `result_valid` asserted for one cycle after an update. |
| **Timing** | Capture on a rising clock edge, update output on the next rising edge (one clock period between these edges). Arithmetic overflow wraps modulo $2^{32}$; no saturation. |

**Q3 – First synthesis and simulations (2 pts)**

**Successful FPGA synthesis (24 October 2026).** We compiled our `mac_core.sv` in **Quartus Prime Lite 18.1**, using the **Cyclone IV E EP4CE22F17C6** (DE0-Nano) as the target device and `mac_core` as the top-level module. The compilation report confirms `Flow Status: Successful`. The synthesis/fit results are:

| FPGA resource | Usage |
|---|---|
| Logic elements | 272 / 22,320 (1%) |
| Registers | 78 |
| Embedded 9-bit multiplier elements | 2 / 132 (2%) |
| Memory bits | 0 / 608,256 |
| Top-level pins | 118 / 154 (77%) |

Table 1. Resource usage reported by Quartus on 24 Oct 2026 (successful compilation).

The relatively high pin count reflects the standalone top-level exposing the input, output, and configuration signals; it does *not* represent a fully integrated board interface. A successful synthesis confirms hardware mapping, **not** correct numerical operation.

**RTL simulation: pending.** We prepared a self-checking `tb_mac_core.sv` testbench for ModelSim/Questa. It will check, among other cases, $3 \times 4 \to 12$, accumulation of $(-2) \times 5 \to 2$, signed shifts, clearing, replace mode, and overflow wrapping. These are **expected values; no passing simulation is claimed yet**. A ModelSim waveform can be added below after execution.

Pending: genuine ModelSim/Questa waveform and testbench pass/fail output.

**Q4 – Perspectives to improve your accelerator (1 pt)**

- **Parallel INT8 MACs:** compute two 8-bit products per 32-bit input word to increase work done per accepted input, with precision trade-offs.
- **Extra pipeline stage:** register the product before accumulation to shorten the combinational critical path and potentially improve clock frequency (at the cost of added latency).
- **Configurable arithmetic:** compare fixed-point scaling and saturation/rounding against the baseline's truncation and wrap-around behavior.
- **Quantitative comparison:** measure latency, throughput, logic elements, embedded multipliers, timing and estimated power after synthesis; compare each optimization with the baseline.

## Report about the use of AI tools

(List uses in accelerator design and report preparation, as requested in the official template.)

**AI tool used:** ChatGPT (OpenAI).

**Uses in the accelerator design:**

- Explanation of the multiply–accumulate operation and the project requirements.
- Discussion and proposal of a baseline architecture (signed 16-bit inputs, 32-bit accumulator, configurable accumulation and shift).
- Assistance in drafting an initial SystemVerilog implementation and a corresponding testbench.
- Guidance on Quartus project setup and interpreting compiler messages; the group corrected the device and top-level settings and obtained a successful synthesis.

**Uses in report preparation:**

- Assistance with technical writing and organizing responses into Q1–Q4 of the official template.
- Generation of a LaTeX draft, including mathematical notation, a block diagram and formatting.

**Verification and responsibility:** The architecture, HDL code, and report text must be reviewed by the student group. Resource figures in Q3 are transcribed from an actual successful Quartus compilation report (24 October 2026). Arithmetic values remain theoretical expectations, not verified simulator outputs. The group will run the testbench and replace the waveform placeholder with an authentic capture before claiming correct circuit operation.

**Student adjustments to disclosure (if applicable):** *[Add or remove uses so that this declaration accurately reflects the group's actual work.]*